ES234317 - Data Structure and Algorithms

Ganjil | 2026-2027

Labwork 03

Module 03: Map, Set, and List

Language: Java

Type: Pre-lab

Mode: individual practical work

Learning outcomes

By completing this module, you should be able to:

1. Use List to store and manage an ordered collection of data.
2. Use Set to store and manage unique data without duplicates.
3. Use Map to store and retrieve data using key-value pairs.
4. Use common operations such as add(), get(), put(), remove(), and contains().
5. Choose the appropriate data structure based on how the data needs to be stored and accessed.
6. Explain how List, Set, and Map can be used to solve different parts of a problem.

Concept Toolkit

Concept	Meaning and a useful distinction
List	A collection that stores elements in a specific order. It allows duplicate elements and elements can be accessed using their index.
Set	A collection that stores only unique elements. Unlike a List, it does not allow duplicate elements.
Map	A collection that stores data as key-value pairs. Each key is unique and is used to access its corresponding value.
add()	Adds an element to a List or Set. If the element already exists in a Set, it will not be added again.
get()	Retrieves an element from a List using its index or retrieves a value from a Map using its key.
put()	Adds a key-value pair to a Map. If the key already exists, its value is replaced.
remove()	Removes an element from a List or Set, or removes a key-value pair from a Map.
contains()	Checks whether a List or Set contains a specific element.
containsKey()	Checks whether a specific key exists in a Map.
size()	Returns the number of elements in a List or Set, or the number of key-value pairs in a Map.

ES234317 - Data Structure and Algorithms
Ganjil | 2026-2027
Labwork 03

Prelab Case:

Problem 1 :
A music application needs a simple program to manage a user's playlist. The songs are recorded in a text file named *playlist.txt*. Each line contains an operation that modifies the playlist.

The playlist must maintain the same order in which songs are added or inserted. Unlike a Set, the same song is allowed to appear more than once in the playlist.

The program must use a List to store and manage all songs in the playlist.

Problem 2 :
A university organizes a programming workshop. Students register for the workshop through an online registration system. Due to repeated submissions, the same student's name may appear more than once in the registration data.

The university wants to know the list of unique participants. The first time a student's name appears determines their position in the final participant list.

The program must use a Set to ensure that every participant appears only once.

Problem 3 :
A small store needs a program to manage its product inventory. Inventory operations are recorded in a text file named inventory.txt.

Each operation contains a product name, an operation type, and a quantity. The same product can appear multiple times in the input data.

The program must use a Map to associate each product with its current stock quantity.

ES234317 - Data Structure and Algorithms

Ganjil | 2026-2027

Labwork 03

Prelab Tasks

A1. Implementation and demonstration

All three problems must be implemented and executed in a single Main.java file.

Problem 1:

Create Java program that manages a user's playlist using a `List<String>`.

1. Read all playlist operations from `playlist.txt` using `Scanner` and process them in their original order.
2. For each operation:
    - `ADD <SONG>` — add the song to the end of the playlist.
    - `INSERT <INDEX> <SONG>` — insert the song at the specified index.
    - `REMOVE <SONG>` — remove the first occurrence of the specified song if it exists. If the song does not exist, do nothing.
3. The same song may appear multiple times in the playlist.
4. After all operations have been processed, display:
    - The total number of songs in the playlist.
    - Every song in its current order, starting from index 0.

Problem 2:

Create Java program that manages workshop participants using a `Set<String>`.

1. Read all registration data from `participants.txt` using `Scanner`.
2. Each line contains one student's name.
3. Add each name to the Set. If the name already exists, do not add it again and increase the duplicate registration counter.
4. Preserve the order in which unique participants first appear in the input.
5. After all registration data has been processed, display:
    - The total number of unique participants.
    - Every unique participant in their first-appearance order.
    - The total number of duplicate registrations.

Problem 3:

Create Java program that manages product inventory using a `Map<String, Integer>`.

1. Read all inventory operations from `inventory.txt` using `Scanner` and process them in their original order.
2. Each operation contains three space-separated values:
   `<TYPE> <PRODUCT> <QUANTITY>`
3. `<TYPE>` can be:
    - `ADD` — add stock to the product.
    - `SELL` — sell stock from the product.
4. For an `ADD` operation:

   - If the product does not exist, create it with the given quantity.
   - If the product already exists, increase its current stock by the given quantity.
5. For a SELL operation:
   - If the product exists and has sufficient stock, subtract the quantity from its stock.
   - If the product does not exist or has insufficient stock, count the operation as a failed sale without changing the stock.
6. Preserve the order in which products first appear in the input.
7. After all operations have been processed, display:
   - The final stock of every product.
   - The total number of failed sales.

A2. Required checks

Input :

Problem 1: **playlist.txt**

```
ADD Ditto
ADD Super Shy
ADD OMG
INSERT 1 Attention
ADD Ditto
REMOVE Super Shy
INSERT 2 Hype Boy
REMOVE Ditto
ADD ETA
```

Problem 2: **participants.txt**

```
Raissa
Fachriza
Dedy
Raissa
Nayla
Fachriza
Tessa
Dedy
Nayla
Rizky
```

Problem 3: **inventory.txt**

```
ADD Laptop 10
ADD Mouse 20
ADD Keyboard 15
SELL Laptop 3
ADD Laptop 5
SELL Mouse 25
ADD Mouse 10
SELL Keyboard 5
SELL Monitor 2
```

Labwork 03

```
SELL Laptop 20
ADD Headset 8
SELL Mouse 3
```

Expected Output

```
===== Problem 1 =====
Total songs: 5
1: Attention
2: Hype Boy
3: OMG
4: Ditto
5: ETA

===== Problem 2 =====
Unique participants: 6
1. Raissa
2. Fachriza
3. Dedy
4. Nayla
5. Tessa
6. Rizky
Duplicate registrations: 4

===== Problem 3 =====
Laptop: 12
Mouse: 27
Keyboard: 10
Headset: 8
Failed sales: 3
```

Submission

Please follow these instructions carefully to submit your work:

1. Create the `lw03` folder under your `dsa-[nrp]` repository. Inside the `lw03` folder, create a subfolder named `prelab`.
2. Complete all of your work for this prelab inside the `prelab` folder.
3. Commit and push your work to GitHub with the commit message: `"Labwork 03 - Prelab"`
4. **Deadline: Monday, October 5, 2026, at 6:00 AM (WIB).**

Labwork 03


dsa-[nrp]/
└── src/
├── lw01/
│ ├── prelab/
│ │ └── [your prelab files]
│ └── unguided/
│ └── [your unguided files]
├── lw02/
│ ├── prelab/
│ │ └── [your prelab files]
│ └── unguided/
│ └── [your unguided files]
├── lw03/
│ ├── prelab/
│ │ └── [your prelab files]
│ └── unguided/
│ └── [your unguided files]
├── ...